



Lab 03

Hàm băm và Chữ ký số

Hash Functions and Digital Signatures

Môn học: **An toàn mạng máy tính**

GVTH: **KS.Nguyễn Ngọc Trưởng**

1 Giới thiệu chung

1.1 Mục tiêu

Bài thực hành này giúp sinh viên làm quen với các khái niệm quan trọng trong lĩnh vực hàm băm mật mã và chữ ký số. Sinh viên sẽ được tiếp cận trực quan với các đặc tính bảo mật của hàm băm, cơ chế xác thực thông điệp và quy trình xác minh chứng chỉ số.

Sau khi hoàn thành bài lab, sinh viên có thể:

1. Hiểu và phân tích các tính chất bảo mật của hàm băm một chiều và MAC.
2. Quan sát và đánh giá tác động của tấn công va chạm trên các hàm băm yếu như MD5 và SHA-1.
3. Sử dụng công cụ và lập trình để tạo giá trị băm và MAC cho thông điệp.
4. Thực hiện xác minh thủ công chứng chỉ số X.509 và chữ ký số RSA.

1.2 Môi trường thực hành

Để thực hiện bài lab này, sinh viên cần chuẩn bị các công cụ và môi trường sau:

- **Máy ảo thực hành:** Cài sẵn công cụ OpenSSL, có thể dùng máy ảo **Google Colab**.
- **Ngôn ngữ lập trình:** Python 3.x hoặc ngôn ngữ tương đương phục vụ cho việc tính toán giá trị băm và xử lý dữ liệu.
- **Kết nối mạng:** Đảm bảo máy ảo có kết nối Internet để tải chứng chỉ số, tập tin mẫu và thực hiện các thao tác với máy chủ thực.

1.3 Chuẩn bị kiến thức

Trước khi bắt đầu bài thực hành, sinh viên cần chuẩn bị các kiến thức nền tảng sau:

- **Kiến thức mật mã học:** Hiểu khái niệm hàm băm, tính một chiều, tính kháng va chạm, MAC và chữ ký số.
- **Kỹ năng lập trình:** Có khả năng xử lý chuỗi, dữ liệu nhị phân, định dạng HEX và thao tác với thư viện băm trong Python.
- **Kiến thức chứng chỉ số:** Đọc trước cấu trúc chứng chỉ X.509, khóa công khai RSA và quy trình xác minh chữ ký số bằng OpenSSL.

2 Nội dung thực hành

2.1 Mã hóa khóa công khai RSA

Mã hóa đối xứng đã phát triển từ cổ điển đến hiện đại nhưng vẫn còn nhiều hạn chế. Hai trong số những vấn đề khó khăn nhất liên quan đến mã hóa đối xứng là:

- **Key distribution** (*Phân phối khóa*) - Vấn đề trao đổi khóa bí mật giữa bên gửi và bên nhận. Phân phối khóa trong mã hóa đối xứng yêu cầu hoặc (1) hai bên liên lạc đã chia sẻ một khóa từ trước, được phân phối bằng một cách nào đó; hoặc (2) sử dụng một trung tâm phân phối khóa. Cần có một kênh an toàn để trao đổi khóa sao cho khóa phải được giữ bí mật và chỉ bên gửi và bên nhận biết. Việc này sẽ khó thực hiện và việc thiết lập một kênh an toàn như vậy sẽ tốn kém và mất thời gian.
- **Digital signatures** (*Chữ ký số*): Nếu việc sử dụng mật mã trở nên phổ biến, không chỉ trong các tình huống quân sự mà còn cho các mục đích thương mại và cá nhân, thì các thông điệp và tài liệu điện tử sẽ cần một thứ tương đương với chữ ký được sử dụng trong tài liệu giấy. Hơn nữa, không có cơ sở để quy trách nhiệm nếu khóa bị lộ.

Diffie và Hellman đã đạt được một bước đột phá đáng kinh ngạc vào năm 1976 bằng cách đề xuất một phương pháp giải quyết cả hai vấn đề trên và hoàn toàn khác biệt so với tất cả các phương pháp tiếp cận mật mã trước đó trong hơn bốn thiên niên kỷ. Đó chính là **mật mã hóa khóa công khai** hay **mật mã hóa bất đối xứng**. Sự phát triển của mật mã khóa công khai là cuộc cách mạng lớn nhất và có lẽ là duy nhất trong toàn bộ lịch sử của ngành mật mã học.

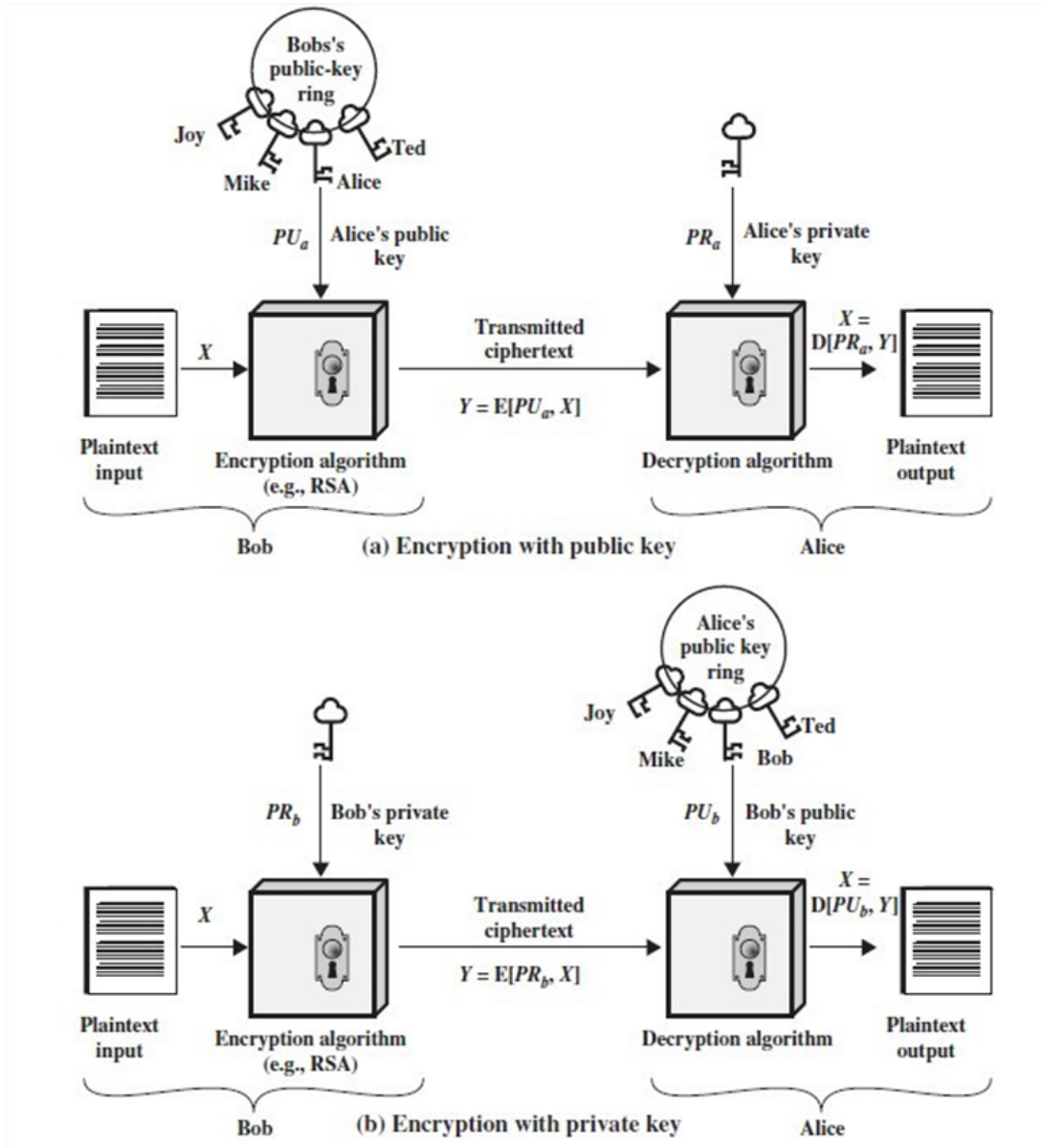
Để phân biệt giữa hai hệ mật mã hiện đại (đối xứng và bất đối xứng), chúng ta gọi khóa được sử dụng trong mã hóa đối xứng là khóa bí mật (secret key). Hai khóa được sử dụng cho mã hóa bất đối xứng được gọi là khóa công khai (public key - PU) và khóa riêng tư (private key - PR). Bản rõ được ký hiệu là M, bản mã được ký hiệu là C. Có 2 hướng tiếp cận chính trong mật mã hóa khóa công khai:

- Hệ mật mã khóa công khai cho tính bảo mật (**Confidentiality**): Nếu Bob muốn gửi một tin nhắn bảo mật cho Alice, Bob sẽ mã hóa tin nhắn bằng khóa công khai của Alice. Khi Alice nhận được tin nhắn, cô ấy sẽ giải mã nó bằng khóa riêng tư của mình. Không có người nhận nào khác có thể giải mã tin nhắn vì chỉ Alice mới biết khóa riêng tư của cô ấy.
 - Mã hóa: $C = E(M, PU)$
 - Giải mã: $M = D(C, PR)$
- Hệ mật mã khóa công khai cho tính xác thực (**Authentication**): Trong trường hợp này, Alice chuẩn bị một tin nhắn cho Bob và mã hóa nó bằng khóa riêng tư của Alice trước khi gửi đi. Bob có thể giải mã tin nhắn bằng khóa công khai của Alice. Bởi vì tin nhắn được mã hóa bằng khóa riêng tư của Alice, chỉ có Alice mới có thể

đã chuẩn bị tin nhắn đó. Do đó, toàn bộ tin nhắn được mã hóa đóng vai trò như một chữ ký số (digital signature).

- Mã hóa: $C = E(M, PR)$

- Giải mã: $M = D(C, PU)$



Hình 1: Two Approaches in Public-Key Cryptography

RSA là một trong những hệ mật mã khóa công khai đầu tiên và được sử dụng rộng rãi cho truyền thông an toàn. Mật mã này được phát triển vào năm 1977 bởi Ron Rivest, Adi Shamir, và Len Adleman tại MIT và được công bố lần đầu vào năm 1978. Trong sơ đồ RSA, the plaintext and cyphertext là các số nguyên từ 0 đến $n - 1$ với một số n nào đó. Kích thước điển hình cho n là 1024 bits hay 309 chữ số thập phân.

Tức là, $n < 2^{1024}$. Thuật toán RSA có thể được tóm tắt như sau:

1. Chọn 2 số nguyên tố lớn p và q , tính $n = p \cdot q$

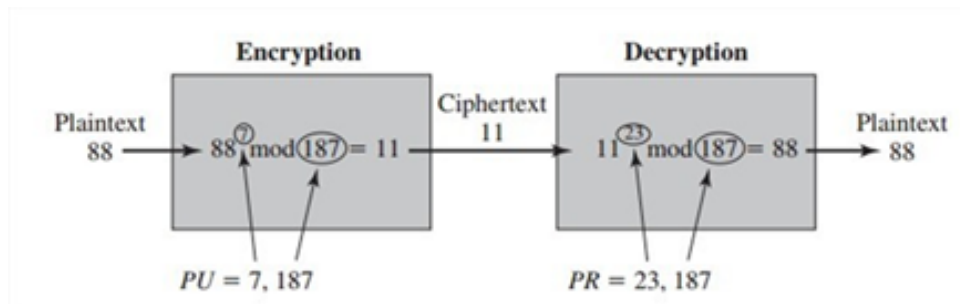
2. Tính $f(n) = (p - 1)(q - 1)$
3. Chọn e sao cho e là số nguyên tố cùng nhau với $f(n)$ và nhỏ hơn $f(n)$.
4. Xác định d sao cho $e.d \equiv 1 \pmod{f(n)}$
(d có thể được tính dựa trên thuật toán Euclid)
5. Các khóa kết quả là khóa công khai $PU = (e, n)$ và khóa riêng tư $PR = (d, n)$

Để mã hóa plaintext M :

- Mã hóa cho tính bảo mật (Confidentiality): $C = E(M, PU) = M^e \bmod n$
- Mã hóa cho tính xác thực (Authentication): $C = E(M, PR) = M^d \bmod n$

Để giải mã ciphertext C :

- Giải mã cho tính bảo mật (Confidentiality): $M = D(C, PR) = C^d \bmod n$
- Giải mã cho tính xác thực (Authentication): $M = D(C, PU) = C^e \bmod n$



Hình 2: Ví dụ về thuật toán RSA

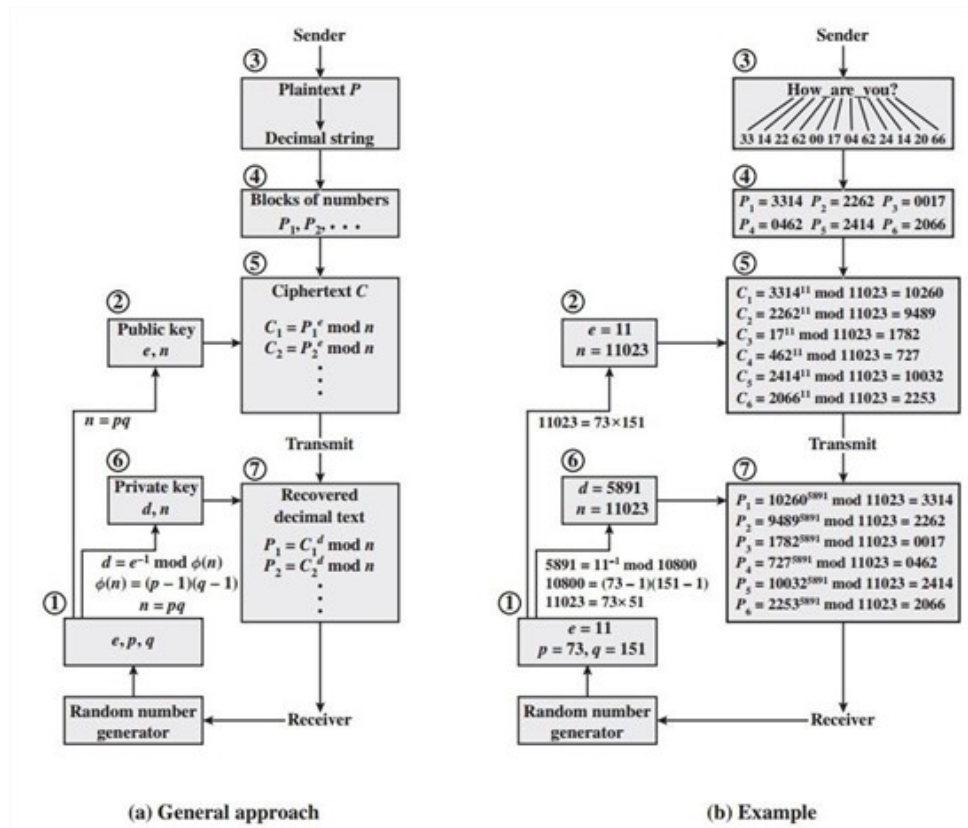
Khi sử dụng thuật toán RSA để xử lý nhiều khối dữ liệu, quy trình bắt đầu bằng việc chuyển đổi thông điệp văn bản thành dạng số. Cụ thể, mỗi ký tự trong bản rõ (plaintext) được gán một mã số thập phân duy nhất gồm hai chữ số. Ví dụ, ký tự 'a' có thể được mã hóa thành 00 và 'A' thành 26. Sau khi chuyển đổi, chuỗi số này được chia thành các khối (blocks), với mỗi khối bao gồm bốn chữ số thập phân, tương đương với hai ký tự. Quá trình mã hóa và giải mã cho nhiều khối dữ liệu được minh họa chi tiết trong Hình 3 của tài liệu gốc, trong đó các số được khoanh tròn chỉ ra thứ tự thực hiện các thao tác.

Nhiệm vụ 2.1

Nhiệm vụ của bạn là viết một chương trình để minh họa cách hoạt động của mật mã ngẫu nhiên nếu p , q , e không được cung cấp. Lưu ý rằng cặp khóa phải "lớn" nhất có thể. Sử dụng các khóa đã tạo để mã hóa/giải mã thông điệp. Thông điệp có thể là số hoặc chuỗi.

1. Xác định khóa công khai PU và khóa riêng PR , nếu:

- $p_1 = 11$, $q_1 = 17$, $e_1 = 7$ (hệ thập phân)



Hình 3: Ví dụ về Xử lý RSA nhiều khối

- $p_2 = 20079993872842322116151219$,
 $q_2 = 676717145751736242170789$,
 $e_2 = 17$ (hệ thập phân)
 - $p_3 = \text{F7E75FDC469067FFDC4E847C51F452DF}$,
 $q_3 = \text{E85CED54AF57E53E092113E62F436F4F}$,
 $e_3 = \text{0D88C3}$ (hệ thập lục phân)
- Sử dụng khóa được tạo ra từ p_1, q_1, e_1 để mã hóa và giải mã bản rõ $M=5$ trong cả hai trường hợp Mã hóa cho tính bảo mật (Encryption for Confidentiality) và Mã hóa cho tính xác thực (Encryption for Authentication).
 - Sử dụng các khóa trên để mã hóa thông điệp sau: The University of Information Technology. Xác định bản mã dưới dạng Base64.
 - Tìm bản rõ tương ứng của mỗi bản mã sau, biết rằng chúng được mã hóa bằng một trong ba khóa đã cho ở trên.
 - $\text{raUcesU1Okx/8ZhgodMoo0Uu18sC20yXlQFevSu7W/FDxIy0YRHMyXcHdD9PBvIT2aUft5fCQEGomiVVPv4I}$
 - $\text{C87F570FC4F699CEC24020C6F54221ABAB2CE0C3}$
 - $\text{Z2BUSkJcg0w4XEpgm0JcMExEQmBIVH6dYEpNTHpMHptMQ7NgT}$

HlgQrNMQ2BKTQ==

- 00101000000101001111111101101110010111011001010111011000110011
110111111001111110110100011001111001100001001010001010100111101
010100110011101110111011110101101100000100

2.2 Đặc tính của hàm băm

Nhiệm vụ 2.2 (*)

Hiện nay, ai cũng biết rằng các hàm băm mật mã MD5 và SHA-1 đã bị bẻ gãy một cách rõ ràng. Chúng ta sẽ tìm hiểu về xung đột MD5 và SHA-1 trong nhiệm vụ này bằng cách thực hiện các bài tập sau:

1. Xem xét hai thôn g điệp (message) dạng HEX như sau:

Message 1

d131dd02c5e6eec4693d9a0698aff95c2fcab58712467eab4004583eb8fb7f8955ad3
40609f4b30283e488832571415a085125e8f7cdc99fd91dbdf280373c5bd8823e315
6348f5bae6dacd436c919c6dd53e2b487da03fd02396306d248cda0e99f33420f577
ee8ce54b67080a80d1ec69821bcb6a8839396f9652b6ff72a70

Message 2

d131dd02c5e6eec4693d9a0698aff95c2fcab50712467eab4004583eb8fb7f8955ad3
40609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd7280373c5bd8823e315
6348f5bae6dacd436c919c6dd53e23487da03fd02396306d248cda0e99f33420f577e
e8ce54b67080280d1ec69821bcb6a8839396f965ab6ff72a70

Có bao nhiêu byte khác biệt giữa hai thông điệp? Hãy tạo giá trị băm MD5 cho mỗi thông điệp và quan sát xem các giá trị MD5 này có giống nhau hay không và mô tả quan sát của bạn trong báo cáo lab.

2. Xem xét hai chương trình thực thi có tên [hello](#) and [erase](#): Chạy các chương trình này từ console và quan sát điều gì xảy ra. Hãy tạo giá trị băm MD5 cho các chương trình này và báo cáo quan sát của bạn.
3. Tải 2 tệp tin PDF: [shattered-1.pdf](#) và [shattered-2.pdf](#). Mở các tệp này để kiểm tra sự khác biệt. Sau đó, tạo băm SHA-1 cho chúng và quan sát kết quả.
4. Rút ra kết luận dựa trên các quan sát của bạn, giải thích lý do cho sự tồn tại của xung đột trong MD5 và SHA-1.

Nhiệm vụ 2.3 (*)

Trong nhiệm vụ này, chúng ta sẽ tạo ra hai tệp khác nhau có cùng giá trị băm MD5. Phần đầu của hai tệp này cần phải giống nhau, tức là chúng chia sẻ cùng một tiền tố (prefix). Chúng ta có thể đạt được điều này bằng cách sử dụng chương trình `md5collgen`, cho phép chúng ta cung cấp một tệp tiền tố với nội dung tùy ý. Cách thức hoạt động của chương trình được minh họa trong hình 4. Thực hành và trả lời các câu hỏi sau:

1. Nếu độ dài của tệp tiền tố của bạn không phải là bội số của 64, điều gì sẽ xảy ra?
2. Tạo một tệp tiền tố có chính xác 64 byte, chạy lại công cụ xung đột và xem điều gì xảy ra.



Hình 4: Tạo MD5 collision từ tiền tố

Gợi ý

Lệnh sau được sử dụng để tạo ra hai tệp đầu ra `out1.bin` và `out2.bin` từ một tiền tố cho trước `prefix.txt`:

```
$ md5collgen -p prefix.txt -o out1.bin out2.bin
```

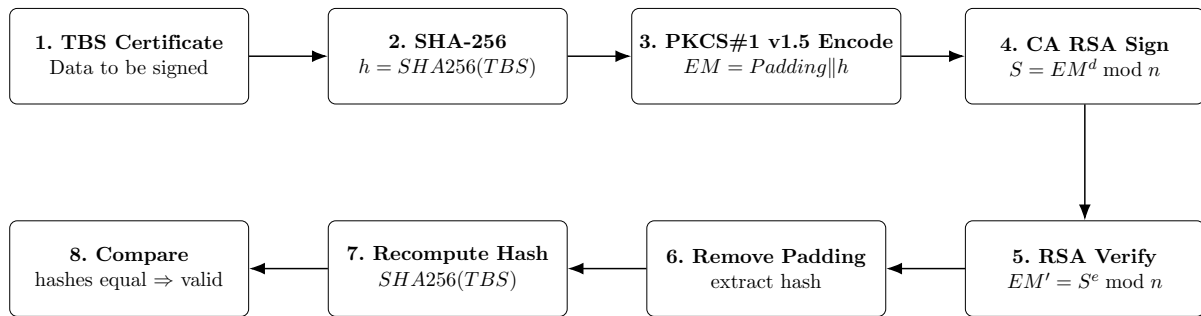
Sau khi tạo xong, ta có thể kiểm tra sự khác biệt giữa hai tệp bằng lệnh `diff`. Ngoài ra, sử dụng `md5sum` để kiểm tra giá trị băm MD5 của từng tệp.

Các lệnh minh họa:

```
$ diff out1.bin out2.bin
$ md5sum out1.bin
$ md5sum out2.bin
$ xxd out1.bin
```

2.3 Xác minh thủ công X.509 Certificate

Trong nhiệm vụ này, chúng ta sẽ xác minh thủ công một chứng chỉ X.509. Một chứng chỉ X.509 chứa dữ liệu về một khóa công khai và chữ ký của nhà phát hành (issuer) trên dữ liệu đó. Chúng ta sẽ tải xuống một chứng chỉ X.509 thực tế từ một máy chủ web, lấy



Hình 5: Quy trình tạo và xác minh chứng chỉ X.509 được đơn giản hóa

khóa công khai của nhà phát hành, và sau đó sử dụng khóa công khai này để xác minh chữ ký trên chứng chỉ.

Bước 1: Tải về một chứng chỉ từ webserver thật

Chúng ta sử dụng máy chủ `www.openssl.org` trong tài liệu này. **Sinh viên chọn một máy chủ web khác có chứng chỉ khác với máy chủ được sử dụng trong tài liệu này.** Chúng ta có thể tải chứng chỉ bằng trình duyệt hoặc sử dụng lệnh sau:

```
$ openssl s_client -connect www.openssl.org:443 -showcerts
```

Kết quả của lệnh chứa hai chứng chỉ. Trường chủ thể (subject) (dòng bắt đầu bằng s:) của chứng chỉ [0] là `www.openssl.org`, tức là đây là chứng chỉ của `www.openssl.org`. Trường nhà phát hành (issuer) (dòng bắt đầu bằng i:) cung cấp thông tin của nhà phát hành. Trường chủ thể của chứng chỉ thứ hai [1] giống với trường nhà phát hành của chứng chỉ đầu tiên [0]. Về cơ bản, chứng chỉ thứ hai thuộc về một CA trung gian (intermediate CA). Trong nhiệm vụ này, chúng ta sẽ sử dụng chứng chỉ của CA để xác minh chứng chỉ của máy chủ.

Sao chép và dán mỗi chứng chỉ (phần văn bản giữa dòng chứa “Begin CERTIFICATE” và dòng chứa “END CERTIFICATE”, bao gồm cả hai dòng này) vào một tệp. Chúng ta hãy gọi tệp đầu tiên là `c0.pem` và tệp thứ hai là `c1.pem`.

Bước 2: Trích xuất public key (e, n) từ chứng chỉ của nhà phát hành

OpenSSL cung cấp các lệnh để trích xuất các thuộc tính nhất định từ chứng chỉ X.509. Chúng ta có thể trích xuất giá trị của n bằng `-modulus`. Không có lệnh cụ thể để trích xuất e , nhưng chúng ta có thể in ra tất cả các trường và có thể dễ dàng tìm thấy giá trị của e .

For modulus (n):

```
$ openssl x509 -in c1.pem -noout -modulus
```

Print out all the fields, find the exponent (e):

```
$ openssl x509 -in c1.pem -text
```

Bước 3: Trích xuất chữ ký từ chứng chỉ của máy chủ

Không có lệnh openssl cụ thể nào để trích xuất trường chữ ký. Tuy nhiên, chúng ta có thể in ra tất cả các trường và sau đó sao chép và dán khối chữ ký vào một tệp (lưu ý: nếu thuật toán chữ ký được sử dụng trong chứng chỉ không dựa trên RSA, bạn có thể tìm một chứng chỉ khác):

```
$ openssl x509 -in c0.pem -text
```

```
...
```

```
Signature Algorithm: sha256WithRSAEncryption
```

```
aa:9f:be:5d:91:1b:ad:e4:4e:ec:8f:07:64:44:35:b4:ad:
```

```
3b:13:3f:c1:29:d8:b4:ab:f3:42:51:49:46:3b:d6:cf:1e:41:
```

```
...
```

```
0e:84:52:94
```

Chúng ta cần loại bỏ dấu cách và dấu hai chấm khỏi dữ liệu, để chúng ta có thể nhận được một chuỗi hex mà chúng ta có thể đưa vào chương trình của mình. Các lệnh sau có thể đạt được mục tiêu này. Lệnh `tr` hỗ trợ công cụ tiện ích của Linux cho các hoạt động chuỗi. Trong trường hợp này, tùy chọn `-d` được sử dụng để xóa “:” và “khoảng trắng” khỏi dữ liệu.

```
$ cat signature | tr -d '[:space:]:'
```

```
84a89a11a7d8bd0b267e52247bb2559dea30895108876fa9ed10ea5b3e0bc7
```

```
...
```

```
5c045564ce9db365fdf68f5e99392115e271aa6a8882
```

Bước 4: Trích xuất nội dung của chứng chỉ máy chủ

Một Nhà cung cấp Chứng thực (CA) tạo chữ ký cho chứng chỉ máy chủ bằng cách đầu tiên tính toán băm của chứng chỉ, sau đó ký lên giá trị băm đó. Để xác minh chữ ký, chúng ta cũng cần tạo băm từ chứng chỉ. Vì băm được tạo trước khi chữ ký được tính toán, chúng ta cần loại trừ khỏi chữ ký của chứng chỉ khi tính toán băm. Tìm ra phần nào của chứng chỉ được sử dụng để tạo băm là khá thách thức nếu không hiểu rõ về định dạng của chứng chỉ.

Chứng chỉ X.509 được mã hóa bằng tiêu chuẩn ASN.1 (Abstract Syntax Notation One), và vì nếu chúng ta có thể phân tích cấu trúc ASN.1, chúng ta có thể dễ dàng trích xuất bất kỳ trường nào từ chứng chỉ. OpenSSL có một lệnh gọi là `asn1parse` được sử

dùng để trích xuất dữ liệu từ dữ liệu định dạng ASN.1, và có khả năng phân tích chứng chỉ X.509 của chúng ta.

```
$ openssl asn1parse -i -in c0.pem
```

```
0:d=0 hl=4 l=1522 cons: SEQUENCE
4:d=1 hl=4 l=1242 cons: SEQUENCE
8:d=2 hl=2 l= 3 cons: cont [ 0 ]
10:d=3 hl=2 l= 1 prim: INTEGER           :02
13:d=2 hl=2 l= 16 prim: INTEGER          :0E64C5FBC236ADE14B172AEB41C78CB0
...
1236:d=4 hl=2 l= 12 cons: SEQUENCE
1238:d=5 hl=2 l= 3 prim: OBJECT           :X509v3 Basic Constraints
1243:d=5 hl=2 l= 1 prim: BOOLEAN          :255
1246:d=5 hl=2 l= 2 prim: OCTET STRING     [HEX DUMP]:3000
1250:d=1 hl=2 l= 13 cons: SEQUENCE
1252:d=2 hl=2 l= 9 prim: OBJECT           :sha256WithRSAEncryption
1263:d=2 hl=2 l= 0 prim: NULL
1265:d=1 hl=4 l= 257 prim: BIT STRING
```

Trường bắt đầu từ ① là phần thân của chứng chỉ được sử dụng để tạo băm; trường bắt đầu từ ② là khối chữ ký. Offset (độ dời) của chứng chỉ là các con số ở đầu dòng. Trong trường hợp của chúng ta, phần thân chỉ là từ offset 4 đến 1249, trong khi khối chữ ký là từ 1250 đến hết tệp. Đối với chứng chỉ X.509, offset bắt đầu luôn giống nhau (tức là 4), nhưng điểm kết thúc phụ thuộc vào độ dài nội dung chứng chỉ. Chúng ta có thể sử dụng tùy chọn `-strparse` để lấy trường từ offset 4, điều này sẽ cho chúng ta phần thân của chứng chỉ, không bao gồm khối chữ ký.

```
$ openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout
```

Khi chúng ta có được phần thân của chứng chỉ, chúng ta có thể tính toán băm của nó bằng lệnh sau:

```
$ sha256sum c0_body.bin
```

Nhiệm vụ 2.4

Thực hiện lại các bước trên để trích xuất tất cả thông tin, bao gồm: khóa công khai của CA, chữ ký của CA và phần thân của chứng chỉ máy chủ. Viết chương trình Python để xác minh chứng chỉ dựa trên công thức RSA.

3 Yêu cầu và đánh giá

Sinh viên được khuyến khích làm việc theo nhóm từ hai đến ba người.

▼ Thêm bài nộp

Nộp tập tin

Kích thước tối đa với một tập tin 10 MB, số lượng tập tin đính kèm tối đa:20

☐	Tên	Sửa lần cuối	Kích thước	Loại
☐	 Lab01_Group02_Report.pdf	13/02/2026 11:05	95.2 KB	Tài liệu PDF
☐	 Lab01_Group02_Resource.zip	13/02/2026 11:06	49.7 KB	Lưu trữ (ZIP)

[Lưu những thay đổi](#) [Hủy bỏ](#)

Hình 6: Yêu cầu bài nộp trên hệ thống UIT Courses

Yêu cầu chung

Bài nộp phải tuân thủ các quy định sau:

- **Bài nộp phải được nộp trên hệ thống UIT Courses** (như hình 6), bao gồm:
 1. Một tệp báo cáo thực hành ở định dạng `LabXGroupY_Report.pdf`, trình bày theo đúng mẫu quy định.
 2. Một tệp `LabXGroupYY_Resource.zip` (nếu có), bao gồm mã nguồn, dữ liệu, chương trình thực thi và các tài nguyên liên quan phục vụ cho việc kiểm tra và đánh giá.
- **Báo cáo thực hành** cần đảm bảo những yêu cầu sau:
 1. Báo cáo phải trình bày đầy đủ nội dung thực hiện, phương pháp, kết quả và các nhận xét cần thiết; nội dung phải rõ ràng, chính xác và có tính hệ thống và hình ảnh minh chứng cần thiết.
 2. Báo cáo không chấp nhận việc sử dụng đồng thời nhiều ngôn ngữ trong cùng một báo cáo, trừ các thuật ngữ kỹ thuật hoặc từ khóa chuyên ngành (cả một câu, cả một đoạn văn).
 3. Đối với các nhiệm vụ lập trình, báo cáo phải mô tả chi tiết các thành phần chính của chương trình, giải thích các bước thực hiện và minh họa kết quả chạy chương trình khi cần thiết. Việc chỉ nộp mã nguồn mà không có mô tả hoặc phân tích sẽ không được chấm điểm cho nhiệm vụ đó.
 4. Bài nộp phải là kết quả làm việc của chính sinh viên hoặc nhóm sinh viên. Sinh viên được phép thảo luận để trao đổi ý tưởng, nhưng mọi hình thức sao chép toàn bộ hoặc một phần báo cáo đều bị xem là vi phạm quy

định về liên chính học thuật và không được đánh giá điểm cho bài thực hành. Mọi tài liệu tham khảo phải được trích dẫn đầy đủ.

- **Nhóm sinh viên phải sử dụng một kho mã nguồn GitHub trong suốt quá trình thực hiện các bài lab.** Các yêu cầu cụ thể như sau:

1. Mỗi bài lab phải được lưu trữ trong kho mã nguồn GitHub và được cập nhật sau khi hoàn thành.
2. Mã nguồn phải được tổ chức theo thư mục tương ứng với từng lab, có cấu trúc rõ ràng và nhất quán.
3. Mỗi thư mục lab phải kèm theo tệp **README.md** mô tả ngắn gọn mục tiêu, nội dung chính và hướng dẫn chạy chương trình (nếu có).
4. Kho mã nguồn phải chứa đầy đủ các phiên bản mã nguồn đã thực hiện trong học phần và duy trì ở trạng thái có thể truy cập được (public hoặc cấp quyền cho giảng viên) để phục vụ việc kiểm tra khi cần thiết.

Lưu ý

Việc đánh giá được thực hiện dựa trên các tiêu chí sau:

- Mức độ hoàn thành của từng nhiệm vụ theo yêu cầu đề bài.
- Tính đúng đắn của kết quả và phương pháp thực hiện.
- Chất lượng trình bày trong báo cáo (rõ ràng, logic, đầy đủ minh họa khi cần).
- Chất lượng mã nguồn (tổ chức, khả năng chạy lại, tính rõ ràng).
- Mức độ tuân thủ các quy định về nộp bài và quản lý mã nguồn.

Đánh giá

- Các Task 2.1, 2.2, mỗi task được tính 2.5 điểm.
- Các Task 2.3, 2.4, mỗi task được tính 2 điểm.
- Các task có dấu (*) là những task được thực hiện tại lớp; nếu nộp bổ sung thì điểm tối đa chỉ đạt 70%.

Lưu ý: Không chia sẻ bất kỳ tài liệu nào của học phần (slide bài giảng, tài liệu đọc, bài tập, bài lab, v.v.) ra ngoài lớp học khi chưa có sự cho phép của giảng viên.

Chúc các bạn học tập tốt và đạt kết quả cao trong học phần!!!